

4. Obsidian 볼트로 학위논문·논문 자료 관리하기

이 문서는 이 저장소 전체(ESE_Lab_template/)를 하나의 Obsidian 볼트(vault)로 사용하는 방법을 안내합니다. Obsidian은 마크다운(.md) 노트를 서로 연결해 아이디어와 자료를 관리하는 도구입니다. LaTeX가 “최종 원고를 조판”하는 도구라면, Obsidian은 그 원고를 쓰기까지의 자료 정리·문헌 노트·연구 일지·구조 설계를 담는 작업 공간입니다.

두 도구는 충돌하지 않습니다. 같은 폴더 안에서 .tex는 LaTeX가, .md는 Obsidian이 담당하며, 서로의 파일을 건드리지 않습니다.

이 저장소는 볼트로 여는 것을 기본 사용 방식으로 권장합니다. 볼트 설정(.obsidian/)이 저장소에 함께 들어 있으므로, 내려받은 폴더를 Obsidian에서 “Open folder as vault”로 열기만 하면 곧바로 문헌 노트·연구 일지·AI 작업 일지가 연결된 상태로 시작합니다. 아래 “볼트 열기”를 참고하세요.

이 저장소가 볼트인 이유

Obsidian에서 “볼트”란 하나의 폴더입니다. 그 폴더 안의 모든 마크다운 파일이 하나의 연결된 노트 묶음이 됩니다. 이 저장소를 볼트로 열면 다음 자료가 한 화면에서 연결됩니다.

- 저장소 루트의 안내 문서(1-..., 2-..., 3-..., 그리고 이 4-... 문서)
- 각 템플릿 폴더의 Readme.md(KNUE_thesis, JKASE/JKESS/RISE_template 등)
- 논문 작업 중 쌓이는 문헌 노트(예: References/저자_연도-제목.md)
- 연구 일지(worklog/ 폴더의 날짜별 작업 기록)

볼트 설정은 .obsidian/ 폴더에 저장됩니다. 이 폴더가 이미 저장소에 들어 있으므로, 아래 “볼트 열기”만 하면 바로 사용할 수 있습니다.

준비물

- Obsidian 설치 (무료, Windows/macOS/Linux). 개인 연구 용도는 무료로 사용할 수 있습니다.

볼트 열기 (Open folder as vault)

1. Obsidian을 실행합니다.
2. 첫 화면 또는 좌측 하단 볼트 전환(Open another vault) → Open folder as vault를 선택합니다.
3. 이 저장소 폴더(...\ESE_Lab_template)를 선택합니다.
4. “Trust author and enable plugins?”를 물으면, 내 저장소이므로 신뢰(Trust)를 선택합니다. (커뮤니티 플러그인을 쓰지 않는다면 굳이 켜지 않아도 됩니다.)

이제 좌측 파일 탐색기에 저장소의 모든 폴더가 보이고, 마크다운 문서를 클릭하면 미리보기와 편집이 됩니다.

한 저장소, 두 도구. VS Code(또는 Overleaf)로는 .tex를 편집·빌드하고, Obsidian으로는 같은 폴더의 .md 노트를 관리합니다. 두 프로그램을 동시에 열어 두어도 됩니다.

왜 논문 작업에 Obsidian을 쓰는가

기능	논문 작업에서의 쓸모
문헌 노트	논문 한 편당 .md 한 개로 요약·인용·메모를 남기고, 서로 연결합니다.
양방향 링크 [[노트]]	"이 개념은 저 논문에서" 같은 연결을 걸어 아이디어 지도를 만듭니다.
그래프 뷰	문헌·개념 사이의 연결을 시각적으로 살펴 연구의 빈틈을 찾습니다.
전체 검색	볼트 전체에서 키워드·인용키·개념을 즉시 찾습니다.
태그 #태그	#읽음 #핵심 #방법론처럼 자료를 분류합니다.
연구 일지(worklog)	날짜별로 무엇을 했고 다음에 무엇을 할지 남깁니다.
아웃라인/구조 설계	장·절 구조와 논증 흐름을 마크다운으로 먼저 잡고 LaTeX로 옮깁니다.

핵심은 “쓰기 전에 생각을 정리하는 공간”입니다. 최종 조판은 LaTeX가 하되, 거기까지 가는 자료·문헌·구상은 Obsidian에 모읍니다.

권장 사용법

1) 문헌 노트 (References)

연구에 참고하는 논문은 **하나의 공용 폴더(저장소 루트의 References/)**에 모으는 것을 권장합니다. 논문 한 편마다 아래 두 파일을 **같은 이름**으로 둡니다.

- **PDF 원문** — APA7 방식으로 저자_연도-논문제목.pdf 로 이름을 정합니다. 파일명은 **70바이트 이내**로 자릅니다(긴 제목은 뒷부분을 줄여도 됩니다. NAS·클라우드 동기화에서 파일명 길이 제한을 피하기 위함입니다). 예: Gummer_2015-Building a Conceptual Framework for Data Literacy.pdf
- **요약 노트** — PDF와 **같은 이름의 .md** 파일. 예: Gummer_2015-Building a Conceptual Framework for Data Literacy.md

PDF와 요약 노트를 같은 이름으로 짝지어 두면, Obsidian에서 노트를 열고 바로 옆의 원문 PDF를 확인할 수 있고, 볼트 검색·그래프·백링크로 문헌을 연결하기 좋습니다. 요약 노트 안에는 서지정보, 핵심 주장, 방법, 인용할 문장, 그리고 .bib의 **인용키(citation key)** 를 적어 두면, 나중에 LaTeX에서 `\parencite{key}` / `\citep{key}`로 바로 연결됩니다.

참고문헌 PDF 이름을 .bib에 맞춰 저자_연도-제목 형태로 자동 정리해 주는 도구가 저장소 루트의 `code/rename_ref_pdfs_by_bib.py`에 있습니다(각 템플릿 Readme의 “참고문헌 PDF 도구” 참고).

```
---
title: "Building a Conceptual Framework for Data Literacy"
authors: [Gummer, Mandinach]
year: 2015
citekey: gummer2015
tags: [데이터리터러시, 이론틀, #핵심]
---
```

한 줄 요약

데이터 리터러시를 구성 요소로 분해한 개념 틀 제안.

인용할 문장

- "..." (p. 12)

관련 노트

- [\[\[Calzada Prado_2013-Incorporating Data Literacy\]\]](#)
- [\[\[내 연구질문\]\]](#)

2) 연구 일지 · AI 작업 일지 (worklog)

worklog/ 폴더에 그날의 작업을 남깁니다. 손으로 쓰는 연구 일지뿐 아니라, **VS Code에서 AI(Claude·GPT·Gemini 등)에게 시킨 작업 내역**도 이곳에 기록하는 것을 권장합니다. 그 잇점과 구체적인 작성법은 아래 AI 작업 일지(worklog) 절을 참고하세요.

3) 원고 구조 설계

장·절 구조와 논증 흐름을 .md로 먼저 잡습니다. 열개가 잡히면 그 순서대로 LaTeX의 sub/...tex 파일에 옮겨 적습니다.

4) 링크와 그래프로 빈틈 찾기

문헌 노트끼리 [\[\[\]\]](#)로 연결하고, 좌측의 **그래프 뷰**를 열어 어떤 주제에 문헌이 몰려 있고 어디가 비었는지 살핍니다. 선행연구의 공백을 시각적으로 확인할 수 있습니다.

추천 설정 (선택)

- **설정 → 파일 및 링크 → 새 링크 형식**: “상대 경로”를 권장합니다(폴더를 옮겨도 링크 유지).
- **설정 → 파일 및 링크 → 첨부 파일 기본 위치**: images/처럼 폴더를 지정하면 붙여넣은 그림이 흩어지지 않습니다.
- **코어 플러그인**: Graph view, Backlinks, Outline, Tag pane, Search를 켜 두면 논문 작업에 충분합니다.
- **커뮤니티 플러그인(선택)**: 서지 관리를 깊게 한다면 Citations 플러그인으로 .bib를 연결해 인용키로 노트를 만들 수 있습니다. 필요할 때만 설치하세요.

AI 작업 일지(worklog) — 왜 남기고, 어떻게 쓰나

요즘은 VS Code 안에서 AI(예: Claude, ChatGPT, Gemini, Cursor)에게 원고 수정·코드 작성·자료 정리를 시키는 일이 많습니다. **그렇게 AI에게 시킨 작업 내역을 worklog/ 폴더에 남기는 것을 강력히 권장합니다.** 이 저장소도 그렇게 관리합니다.

잇점

- **세션 복구**. AI와의 대화 맥락은 세션이 끝나면 사라집니다. worklog가 있으면 다음 세션(또는 다른 AI)에서 “지난번에 무엇을 어디까지 했는지”를 즉시 이어갈 수 있습니다.
- **추적성**. 어떤 AI(회사·모델)가, 어떤 요청을 받아, 어떤 파일을 왜 바꿨는지가 남습니다. 몇 주 뒤 “이 부분이 왜 이렇게 됐지?”를 되짚을 때 근거가 됩니다.
- **검증·책임**. AI가 바꾼 내용을 사람이 검토하는 기준점이 됩니다. AI는 그럴듯하지만 틀린 변경을 하기도 하므로, 무엇이 바뀌었는지 기록해 두면 확인이 쉽습니다.
- **여러 AI 병행**. 여러 회사·모델을 섞어 쓸 때 누가 무엇을 했는지 구분됩니다.
- **연구윤리**. 논문에서 요구하는 **생성형 AI 사용 명시**(어떤 모델을 어디에 썼는지)를 그대로 근거 자료로 쓸 수 있습니다.
- **지식 축적**. 잘 통한 프롬프트·작업 절차가 쌓여 다음에 재사용됩니다.
- **Obsidian 연결**. worklog 노트에서 바꾼 파일을 [\[\[파일노트\]\]](#)로 링크하면, 그래프에서 “언제 어떤 파일을 손냈는지”를 시각적으로 따라갈 수 있습니다.

작성 방법

1. worklog/ 폴더에 YYYY-MM-DD_HH-mm.md 파일을 만듭니다(예: 2026-07-05_20-56.md).
2. 머리말(프론트매터)에 **사용한 AI 신원**을 적습니다 — 에이전트/회사/모델, 그리고 보조 로컬 모델이 있으면 함께.
3. 본문은 **요청사항** → **실행사항** → **결과**를 표로 정리하고, 필요한 상세를 덧붙입니다. 한국어로 작성합니다.
4. 바뀐 주요 파일은 [[JKASE_main]]처럼 노트 링크로 연결해 두면 추적이 쉽습니다.

가장 쉬운 방법: 작업이 끝날 때 AI에게 그대로 부탁하세요 — “오늘 작업 내역을 worklog에 남겨줘.” 그러면 AI가 위 형식에 맞춰 로그 파일을 만들어 줍니다.

형식과 템플릿의 자세한 규칙은 [worklog/README.md](#)에 있습니다.

AI 도구를 쓰려면 Python 환경부터 준비하세요

논문을 AI의 도움으로 작성·분석하려면, 먼저 **Python**을 설치하고 **가상환경(conda 등)**을 구성해 두는 것이 좋습니다. Claude Code·GPT Codex·GitHub Copilot 같은 도구는 VS Code와 Python 환경이 갖춰져 있을 때 코드 실행·자료 분석·그림 생성까지 매끄럽게 도와줄 수 있습니다. 설치와 가상환경 구성, 그리고 각 AI 도구에 코드를 요청하는 방법은 [4-PYTHON_CONDA_VSCODE_SETUP.md](#)를 참고하세요.

Claude Code를 Windows VS Code PowerShell에서 설치하고 사용하는 자세한 방법은 [6-CLAUDE_CODE_SETUP.md](#)를 참고하세요.

Git 으로 기록 남기기 (커밋)

Obsidian이 “지금의 생각”을 담는다면, **Git**은 “**시간에 따른 변화**”를 담습니다. 원고·노트·템플릿을 Git으로 커밋해 두면 언제든지 과거 상태로 되돌리고, 무엇이 바뀌었는지 비교할 수 있습니다. 이 저장소는 이미 Git 저장소이므로 바로 커밋할 수 있습니다.

준비

- Git 설치(설치 안내는 [3-VSCODE_TeXLive_SETUP.md](#) 참고).
- (선택) GitHub 계정 — 원격 백업·협업이 필요할 때.
- 빌드 부산물(.aux, .log, .pdf 등)과 Obsidian 개인 설정은 이미 .gitignore로 제외되어, 원고 소스와 노트만 깔끔하게 기록됩니다.

기본 흐름 (터미널)

```
git status          # 무엇이 바뀌었는지 확인
git add -A          # 바뀐 파일 모두 스테이징 (특정 파일만: git add 파일명)
git commit -m "서론 수정, 표 2 추가" # 변경 이유가 드러나는 메시지로 커밋
git push            # (원격이 있으면) GitHub 등에 업로드
```

VS Code를 쓰면 왼쪽 **소스 제어(Source Control)** 탭에서 같은 일을 버튼으로 할 수 있습니다.

커밋 메시지 요령

- **무엇을 왜** 바꿨는지 한 줄로: “심사 의견 반영: 방법 절 표본 수 정정” 처럼.
- 자주, 작게 커밋할수록 나중에 되돌리거나 비교하기 쉽습니다.

되돌리기

```
git restore 파일명      # 아직 커밋 안 한 변경을 되돌림(주의: 변경 사라짐)
git log --oneline        # 과거 커밋 목록 보기
git revert <커밋해시>    # 특정 커밋을 취소하는 새 커밋 만들기(안전)
```

제출 시점을 커밋(태그)해 두면 → 나중에 “수정본”만 뽑아낼 수 있습니다

이것이 Git과 원고 작업을 함께 쓰는 가장 큰 이유입니다. 논문을 제출하는 순간의 상태를 커밋하고 태그를 달아 두면, 이후 수정한 내용만 골라내어 심사자에게 보여줄 “수정 표시본”을 자동으로 만들 수 있습니다.

1. 제출 시점에 태그를 겁니다.

```
git add -A && git commit -m "JKASE 투고본 제출"
git tag submitted-v1      # 제출본에 이름표를 붙임 (원하는 이름 사용)
```

2. 심사 의견을 반영해 계속 수정하고 커밋합니다. (평소처럼 git commit)

3. 제출 이후 바뀐 부분만 추출합니다. 두 가지 방법이 있습니다.

- 변경 목록·요약 보기:

```
git diff submitted-v1 -- '*.tex'      # 제출본 대비 무엇이 바뀌었는지
git log --oneline submitted-v1..HEAD  # 제출 이후의 커밋(수정 이력) 목록
```

- 수정 표시 PDF 만들기(추가·삭제가 색으로 보이는 원고): 학위논문 템플릿 (KNUE_thesis/)에는 이를 자동화한 make-diff 스크립트가 들어 있습니다. 제출 태그와 현재 원고를 latexdiff로 비교해, 추가된 곳은 파란색, 삭제된 곳은 빨간 취소선으로 표시한 PDF를 만들어 줍니다.

```
# 예: KNUE_thesis 폴더에서 (Windows: make-diff.cmd)
./make-diff.sh submitted-v1 HEAD
```

이렇게 하면 “무엇을 어떻게 고쳤는지”를 심사자에게 그대로 보여줄 수 있어, 재심사·수정본 제출에 특히 유용합니다. 학위논문의 심사위원별 색상 수정 표시 기능과도 함께 쓸 수 있습니다 (자세한 사용법은 [KNUE_thesis/Readme_KNUethesis.md](#) 참고).

핵심: 제출할 때마다 커밋+태그(submitted-v1, submitted-v2, ...)를 남겨 두면, 각 심사 라운드 사이에 무엇이 바뀌었는지 언제든 되짚고 수정 표시본을 뽑아낼 수 있습니다. 원고를 손대기 전에 “제출 시점 태그”부터 남기는 습관을 들이세요.

LaTeX와 함께 쓸 때 주의점

- .tex / .cls / .bib 파일은 Obsidian에서 편집하지 마세요. Obsidian은 마크다운 전용입니다. 이 파일들은 VS Code나 Overleaf에서 다룹니다. (Obsidian에 그대로 보이긴 하지만 편집·조판 도구가 아닙니다.)
- 빌드 부산물(.aux, .log, .pdf 등)은 노트가 아닙니다. 그래프·검색을 어지럽히면 Obsidian 설정 파일 및 링크 → 제외된 파일(Excluded files)에 *.aux 등을 추가하거나, 각 템플릿 폴더의 latexmk/빌드 스크립트로 정리하세요. 이 부산물은 이미 .gitignore로 Git 추적에서 제외됩니다.
- 볼트 설정 공유. .obsidian/의 공유 설정(테마·플러그인 목록·그래프 설정)은 Git에 커밋되고, 개인 창 레이아웃(.obsidian/workspace.json)과 캐시는 .gitignore로 제외됩니다. 다른 PC에서 이 저장소를 내려받아 볼트로 열면 같은 기본 설정으로 시작합니다.

요약

1. Obsidian 설치 → **Open folder as vault**로 이 저장소를 엽니다.
2. .tex는 VS Code/Overleaf, .md는 Obsidian이 담당합니다(서로 안 건드림).

3. **문헌 노트 + 연구 일지 + 구조 설계**를 Obsidian에 모으고, 최종 원고는 LaTeX로 조판합니다.
4. 인용키(citekey)를 노트에 적어 두면 `.bib` → `\citep{key}`로 매끄럽게 이어집니다.